

1 General PPO Framework and Continuous-Control Validation

1.1 Motivation

The earlier PPO implementation was coupled to the custom grid-world: the agent memory stored `CellBox` objects, the policy assumed a discrete action list, and the update processed one trajectory at a time. Since the final ROSbot task uses continuous velocity commands, the framework was refactored into a general-purpose PPO implementation that depends only on the Gymnasium interface.

The refactored framework supports

- discrete and continuous action spaces,
- vectorised environments with correct handling of episode boundaries,
- minibatch PPO updates with several optimisation epochs,
- pluggable observation and reward functions through wrappers.

To verify that the framework is independent of the grid-world, it was tested on the continuous-control benchmark `InvertedPendulum-v5`.

1.2 Network Architecture

The policy and value function share one network body. The body encodes the observation o into a hidden representation h :

$$h = f_{\text{body}}(o).$$

The body consists of an input linear layer with ReLU activation, followed by L residual-style blocks. Each block applies

$$\text{Block}(x) = \text{ReLU}(\text{LayerNorm}(Wx + b)).$$

Two heads read from h :

- the *critic head*, a linear layer producing the scalar value $V_{\theta}(o)$ without output activation;
- the *actor head*, which depends on the action space.

For discrete actions the actor produces a categorical distribution,

$$\pi_{\theta}(a \mid o) = \text{softmax}(Wh + b).$$

For continuous actions the actor produces a diagonal Gaussian,

$$\pi_{\theta}(a \mid o) = \mathcal{N}(\mu_{\theta}(o), \sigma^2), \quad \mu_{\theta}(o) = Wh + b, \quad \sigma = \exp(\log \sigma_{\theta}),$$

where $\log \sigma_\theta$ is a state-independent learnable parameter vector, one entry per action dimension. For a multidimensional action the log probability of the complete action is the sum over dimensions,

$$\log \pi_\theta(a \mid o) = \sum_{j=1}^{d_a} \log \pi_\theta(a_j \mid o),$$

and the entropy is summed in the same way.

All linear layers use orthogonal initialisation with gain $\sqrt{2}$. The actor output layer uses gain 0.01, so that the initial policy is close to uniform (discrete case) or has near-zero mean (continuous case), which favours early exploration. The critic output layer uses gain 1.

1.3 PPO Objective

The implementation uses the clipped surrogate objective. With the probability ratio

$$\rho_t(\theta) = \exp \left(\log \pi_\theta(a_t \mid o_t) - \log \pi_{\theta_{\text{old}}}(a_t \mid o_t) \right),$$

the actor loss is

$$L_\pi = -\mathbb{E}_t \left[\min \left(\rho_t A_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) A_t \right) \right].$$

The critic loss is the mean squared error to the return target,

$$L_V = \mathbb{E}_t \left[(V_\theta(o_t) - \hat{R}_t)^2 \right],$$

and the entropy term encourages exploration. The total loss is

$$L = L_\pi + c_v L_V - c_e \mathbb{E}_t \left[\mathcal{H}(\pi_\theta(\cdot \mid o_t)) \right]$$

with $c_v = 0.5$ and $c_e = 0.01$. Gradients are clipped to a maximum norm of 0.5 before every optimiser step.

1.4 Advantage Estimation and Bootstrapping

Advantages are computed with Generalised Advantage Estimation (GAE). The temporal-difference residual is

$$\delta_t = r_t + \gamma (1 - \text{terminated}_t) V(o_{t+1}) - V(o_t),$$

and the advantage is accumulated backwards,

$$A_t = \delta_t + \gamma \lambda (1 - d_t) A_{t+1}.$$

Here d_t marks the end of a GAE segment. The value targets and the normalised advantages are

$$\hat{R}_t = A_t + V(o_t), \quad \tilde{A}_t = \frac{A_t - \bar{A}}{\text{std}(A) + 10^{-8}}.$$

Two details are important for correctness.

1. **Terminated versus truncated.** The bootstrap value is set to zero only when the episode genuinely terminated. If the episode was cut by a time limit (**truncated**), the critic estimate $V(o_{t+1})$ is still used, because the true return is not zero.
2. **Segment boundaries.** The last step of every rollout is marked as a GAE boundary, so advantages never propagate across two rollouts. This is valid because δ_t already contains the bootstrap term for the next observation.

1.5 Vectorised Rollout Collection

Training uses Gymnasium’s `SyncVectorEnv` with the `SAME_STEP` autoreset mode. In this mode, when an environment finishes, the observation returned by `step()` is already the first observation of the next episode. The true final observation is available in `info["final_obs"]`. The training loop therefore builds the next-state value from

$$o_{t+1}^{\text{real}} = \begin{cases} \text{final_obs}, & \text{if the environment finished at step } t, \\ o_{t+1}, & \text{otherwise.} \end{cases}$$

Without this correction, the value of the first observation of a new episode would be used as the bootstrap target of the last step of the old episode.

The rollout is stored as tensors of shape $(T, N, \dots)$ with T the rollout length and N the number of environments. Before the update it is rearranged to environment-major order $(N \cdot T, \dots)$, so that every environment’s trajectory is contiguous, which the backward GAE pass requires. For continuous actions, sampled actions are clipped to the action-space bounds before being passed to the environment.

1.6 Minibatch Update

Each rollout contains $B = N \cdot T$ transitions. The update shuffles all transitions and processes them in minibatches of size m for K epochs:

rollout $\rightarrow$ GAE and returns $\rightarrow$ normalise advantages
 $\rightarrow$ shuffle $\rightarrow$ minibatch gradient steps (K epochs).

With $B = 2048$, $m = 256$ and $K = 10$, one PPO update performs $8 \times 10 = 80$ gradient steps.

1.7 Pluggable Observation and Reward Wrappers

Two wrappers keep the observation and the reward independent of the environment and the algorithm.

`ObservationCustom` is a Gymnasium `ObservationWrapper`. It applies a state-to-vector function ϕ and derives the new `Box` observation space from the output shape of ϕ .

`RewardCustom` replaces the environment reward by a user-supplied function. Instead of imposing a fixed signature, it inspects the function’s parameters with `inspect.signature` and passes only the context variables that the function requests (or all of them if the function accepts `**kwargs`). The available variables are `state`, `action`, `next_state`, `obs`, `next_obs`, `reward`, `terminated`, `truncated`, `info`, and `env`. The original reward is kept in `info["original_reward"]`.

As an illustration, a reward function that penalises the pole angle only needs to declare the variables it uses:

```
def my_reward(obs, next_obs, reward, terminated):
    return -abs(obs[2]) + 0.1 * reward
```

This is the interface through which the planned LLM component will inject generated reward functions, without changing the environment or the PPO code.

Environments are replicated for vectorised training by deep-copying a template environment (wrappers included), so every sub-environment owns an independent copy. A sanity check after `reset(seed=0)` verifies that the sub-environments are distinct objects and do not share random-number state.

1.8 Experimental Setup

The framework was tested on `InvertedPendulum-v5`, a continuous-control task with a four-dimensional observation (cart position, pole angle, and their velocities) and a one-dimensional continuous action (cart force). The reward is +1 for every step in which the pole remains upright, and the episode is truncated after 1000 steps. The maximum achievable return is therefore 1000.

For this experiment the observation and reward functions were the identity, i.e. the native environment reward was used.

Parameter	Value
Environments in parallel (N)	8
Rollout length (T)	256
Samples per update ($N \cdot T$)	2048
Total environment transitions	500,000
Hidden dimension	64
Residual blocks	2
Learning rate (Adam)	3×10^{-4}
Discount γ	0.90
GAE parameter λ	0.95
Clipping coefficient ϵ	0.2
Value-loss coefficient c_v	0.5
Entropy coefficient c_e	0.01
Epochs per update (K)	10
Minibatch size (m)	256
Gradient-norm clip	0.5

Table 1: PPO configuration for `InvertedPendulum-v5`.

Since $500,000/2048 \approx 244.1$, training performed 245 PPO updates and 501,760 environment transitions.

1.9 Results

After training, the policy was evaluated on a single non-vectorised environment for ten episodes. The final reported episode reached a total score of

$$\boxed{1000},$$

which equals the maximum possible return: the pole stayed balanced until the episode was truncated by the time limit (`truncated = True, terminated = False`). The logged observations show that the pole angle stays small throughout the episode and that the agent applies forces of alternating sign, which is the expected correcting behaviour.

This result confirms that

- the continuous (Gaussian) action path works end to end,
- the GAE computation with episode boundaries produces usable advantages,
- the vectorised training loop with `SAME_STEP` autoreset converges on a standard benchmark.

1.10 Limitations and Next Steps

The result should be interpreted as a validation of the framework, not as a claim about the final navigation task.

1. **Evaluation protocol.** The evaluation samples actions from the stochastic policy and does not clip them to the action bounds (some logged actions lie outside $[-3, 3]$, the range of the environment). The environment limits the applied force, but a deterministic evaluation with clipped actions, reported as mean and standard deviation over all ten episodes and several training seeds, would be a stronger statement.
2. **No training curve.** Only the final score is reported; episode return over training steps should be logged.
3. **Task difficulty.** `InvertedPendulum-v5` is among the easiest MuJoCo tasks; it validates correctness but not performance on harder problems.
4. **Identity wrappers.** The observation and reward functions were identity functions in this run. The reward wrapper has therefore been implemented but not yet exercised with a non-trivial reward.
5. **Code cleanup.** The old standalone `compute_gae` (without done flags) is unused and should be removed. The base `Agent.learn` still accesses the grid-world attribute `.name` and is only safe because `Agent_PPO` overrides it.

The next stage is to connect the LLM reward compiler to `RewardCustom` and to compare LLM-generated rewards against hand-crafted rewards, first on the grid-world and then on the ROSbot navigation environment.

1.11 Conclusion

The PPO implementation was generalised from a grid-world-specific implementation to a framework that works with any Gymnasium environment with discrete or continuous actions. It combines a shared actor-critic network, a Gaussian policy for continuous control, GAE with correct termination and truncation handling, minibatch updates, and vectorised rollouts with SAME_STEP autoreset. On `InvertedPendulum-v5` the trained agent reached the maximum return of 1000 after about 5×10^5 transitions. The observation and reward wrappers provide the integration point for the LLM-generated rewards that form the core of the thesis.